

ABM Platform — V1 Production Vision

Status: Locked after xlsx + transcript review, 5 rounds of Q&A, and engagement-system reverse-engineering.
Supersedes: V1_VISION.md (kept as historical reference). **Owner:** Sunny Dsouza • **Primary user:** Galyna • **Date:** May 22, 2026

TL;DR

Replace Galyna's 16-spreadsheet workflow with **one internal web app** that runs the full ABM pipeline as 8 connected stages: **ingest** → **score** → **asset-link** → **activate** → **engage** → **bucket** → **convert** → **report**. Scoring works today (the CLI). The remaining 7 stages are what V1 builds, with hard production scaffolding (Postgres, Arq workers, Clerk auth, audit log, CI/CD, Sentry) from day one. No vibe-coding — every stage has typed contracts, every action is audited, every external integration is a pluggable connector.

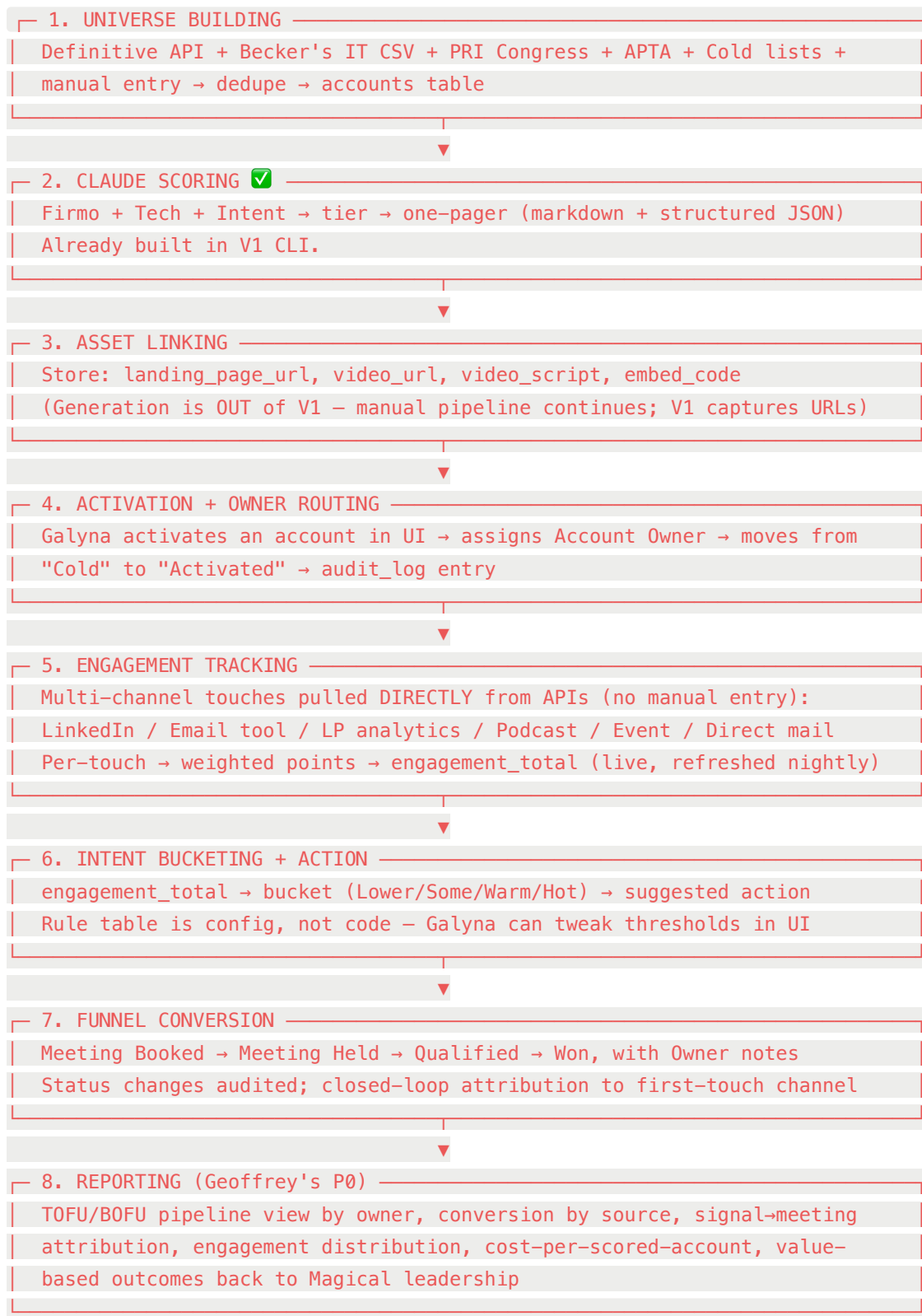
What changed from V1_VISION.md (and why this rewrite was needed)

The first vision doc was correct about the scoring engine but missed that the xlsx is *the entire ABM operating system* — not just an output of scoring. Specifically:

Gap in V1_VISION.md	What the xlsx + transcript revealed
Single "scoring" stage	The xlsx shows two separate scoring systems running in parallel — Claude account-fit scoring (/30) AND engagement scoring (channel touchpoints)
No asset layer	Every scored account has a landing page + personalized Vanessa video + embed code already in production
No activation routing	Every activated account has an Account Owner (Justin / Stephen / Aidan / Tyler / Matt / Colin) — there's a human routing layer
No multi-channel engagement	10 channels with weighted point values are tracked today; Total Score = $\Sigma(\text{touches} \times \text{weights})$
No intent bucketing	Score → bucket (Lower/Some/Warm/Hot) → action (Nurture/Light/Active/Meeting-ask) is a real production decision rule
No funnel conversion	Meeting Booked → Held → Qualified is the closing layer
Reporting as "Phase 5+ later"	Geoffrey Martin and Marco called reporting P0 explicitly in the May 22 sync — it's a primary V1 surface

This doc fixes all of that.

The 8-stage pipeline



Stage-by-stage specification

Stage 1 — Universe Building

Inputs: Definitive Health API (primary), Becker's IT CSV, PRI Congress CSV, APTA CSV, manually curated rep lists (JP/Ahmed/Matt/Colin/JG/Alys), manual UI add.

Connector pattern: every source implements:

```
class AccountConnector(Protocol):
    source_name: str
    def fetch() -> Iterable[RawAccount]: ...
    def normalize(raw: RawAccount) -> NormalizedAccount: ...
```

Dedupe key: `(normalize(name), segment)` — case-insensitive, strips legal suffixes (LLC, Inc, Health System). FKA/AKA variants merged.

Customer flag: if account already a Magical customer (per CRM cross-ref), tag and filter out of cold pipeline.

Output: rows in `accounts` table with `source` and `segment` set. Triggers Stage 2 scoring job.

Stage 2 — Claude Scoring (already built)

What V1 CLI does today, lifted into the platform: - Loads segment prompt → Claude Opus 4.7 + `web_search` → markdown report - Parses `ScoredAccount` structured JSON - Writes `scoring_runs` row with `raw_markdown`, `structured_json`, `cost`, `tokens`, `tier` - Updates `accounts.current_score / current_tier / last_scored_at`

Retrigger: manual UI button, new signal arrival (Tier 1/2 only), nightly batch for stale (>30 days) Tier 1 accounts.

Stage 3 — Asset Linking

V1 stores, does not generate: - `landing_page_url` (e.g. `getmagical.com/abm/anderson-orthopaedic-clinic`) - `video_url` (e.g. `video.gan.ai/<id>`) - `video_script` (full personalized text — Claude can draft a v0 in V1.5 if asked) - `embed_code` (iframe HTML)

How URLs arrive in V1: - Set manually in UI when activating an account - Bulk-imported from existing xlsx as part of one-time migration - Future: webhook from gan.ai render-complete event (V2)

Why: the LP + video pipeline already works; adding generation to V1 would 3x the scope. V1 just makes the URLs first-class data.

Stage 4 — Activation + Owner Routing

Activation flow: 1. Galyna reviews scored account in UI 2. Clicks "Activate" → must pick Account Owner from dropdown (Justin / Stephen / Aidan / Tyler / Matt / Colin / new-owner-add) 3. Optionally: set `channel_first_touch` (TOFU / BOFU / Email / Event / Other) 4. `accounts.is_activated=true`, `accounts.owner_id=<id>`, `audit_log` row 5. Owner gets a Slack DM (V1.5) with one-pager link

Routing logic V1: manual selection. No auto-assignment rules. Galyna decides. **Routing logic V2:** suggest owner based on segment / region / current load — model the field now, build the suggestion engine later.

Channel (1st touch) is captured at activation for attribution.

Stage 5 — Engagement Tracking (the new heavy thing)

Source-of-truth: APIs, not manual entry. Galyna does not log touches.

Pluggable connector pattern:

```
class EngagementConnector(Protocol):
    channel: ChannelType
```

```
def poll() -> Iterable[EngagementTouch]: ...
```

Each connector returns:

```
class EngagementTouch(BaseModel):
    account_id: int
    channel: ChannelType          # see weights table below
    touch_count: int              # how many touches this poll
    content_ref: str | None       # piece title, email subject, event name
    observed_at: datetime
    raw_payload: dict             # source-specific blob
    fingerprint: str             # dedupe key
```

Channels + weights (from xlsx legend):

Channel	Weight	Source of data	API status
BOFU touch	10	Internal content engagement (BOFU pieces)	TBD per Sunny's access
Response (agreed to meeting)	10	CRM (Salesforce or Apollo Sequences)	TBD
TOFU touch	6	Internal content engagement (TOFU pieces)	TBD
Podcast Guest	4	Manual flag (or Buzzsprout API if applicable)	TBD
Event Attend	4	Event platform API or manual flag	TBD
Direct Mail Response	4	Manual flag (no public API for direct mail)	Manual exception
LP Visit	2	LP hosting analytics API	TBD
LP Video View	2	gan.ai analytics API	TBD
LinkedIn Content engagement	2	LinkedIn Sales Nav / Company API / Apify scraping	TBD
LinkedIn Connect (accepted)	2	LinkedIn Sales Nav API	TBD
Email Engagement (open/click)	1	Apollo Sequences or Outreach API	TBD

APIs Sunny is sending access to — connectors built one-at-a-time per phase.

Compute:

```
engagement_total = sum(touch.count × weights[touch.channel]
                        for touch in account.engagement_touches)
```

Computed on write (when new touches land) and stored on `accounts.engagement_total` for fast filtering.

Underlying touches preserved in `engagement_touches` table for re-computation when weights change.

Refresh cadence: each connector polls hourly or on webhook (whichever the API supports). Engagement totals recompute on touch insert.

Stage 6 — Intent Bucketing + Action

Rule table (from xlsx legend, lives in `intent_thresholds` table):

Min score	Max score	Bucket	Action
0	5	Lower	Keep in nurture
6	11	Some	Light SDR/AE follow-up
12	20	Warm	Sales active outreach
21	∞	Hot	Immediate Sales outreach + meeting ask

Editable via UI — Galyna can tune cutoffs without code changes. Edits go to `audit_log`.

When an account crosses a bucket boundary upward (e.g. Warm → Hot), trigger: - Slack DM to Owner with "Account just turned Hot" - Re-score in background (if scoring run is >7 days old) - Suggested-action card appears in Owner's triage

Stage 7 — Funnel Conversion

Status field on each account row:

`ACTIVATED → MEETING_BOOKED → MEETING_HELD → QUALIFIED → WON | LOST | PARKED`

Each transition: - Captured in `account_status_changes` table (status, from, to, owner_id, at, note) - Updates `accounts.current_status` - Optionally writes back to Salesforce (V2)

Comments / notes stored in `account_notes` table — append-only, owner-attributed.

Stage 8 — Reporting (PO per Geoffrey)

Three first-class report surfaces in V1:

8a. Galyna's Triage Dashboard (Monday morning) - "What's new this week" feed (signals + new Tier 1 + bucket changes) - 4 hero stats: New Tier 1 / Score-ups / Score-downs / Unprocessed signals

8b. Owner Pipeline View (per Owner, plus all-owners) - Activated accounts by status (Activated / Meeting Booked / Held / Qualified / Won) - Engagement total + bucket per account - Days since last touch - Filter by segment, source, bucket

8c. Attribution + ROI Report (leadership view) - Conversion rate by `channel_first_touch` - Conversion rate by `source` (Definitive / Becker's / PRI / APTA / cold lists) - Engagement → meeting cycle time - Cost per scored account (Anthropic API spend) - "Value-based reporting" — when an account converts to Won, show revenue uplift attributed (manual entry V1, automated V2)

All three rendered via Next.js pages; underlying queries against Postgres views. No Omni / Looker until V2.

Data model — production schema

-- — Universe —

```
CREATE TABLE accounts (
```

```

id          BIGSERIAL PRIMARY KEY,
name        TEXT NOT NULL,
normalized_name TEXT NOT NULL,    -- for dedupe; case-insensitive, stripped
segment     TEXT NOT NULL CHECK (segment IN ('specialties','payer','hs')),
domain      TEXT,
is_customer  BOOLEAN NOT NULL DEFAULT FALSE, -- already Magical customer
is_activated BOOLEAN NOT NULL DEFAULT FALSE,
owner_id     BIGINT REFERENCES users(id),
channel_first_touch TEXT,        -- TOFU | BOFU | Email | Event | Other
current_status TEXT NOT NULL DEFAULT 'COLD',
-- COLD | ACTIVATED | MEETING_BOOKED | MEETING_HELD | QUALIFIED | WON | LOST | PARKED
current_score NUMERIC(5,2),      -- Claude score
current_tier  TEXT,              -- High Fit / Medium Fit / Low Fit / Tier 1 / etc
current_max   INTEGER,
engagement_total NUMERIC(8,2) NOT NULL DEFAULT 0,
intent_bucket TEXT,             -- Lower | Some | Warm | Hot
intent_action TEXT,            -- the action label for the current bucket
last_scored_at TIMESTAMPTZ,
last_touch_at  TIMESTAMPTZ,
first_seen_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
source         TEXT NOT NULL,    -- definitive | beckers | pri | apta | manual | ...
source_list     TEXT,           -- "Cold Ortho" | "Cold List" | "JG list" | etc.
UNIQUE (normalized_name, segment)
);

```

```
-- ——— Contacts (decision makers) ———
```

```

CREATE TABLE contacts (
id          BIGSERIAL PRIMARY KEY,
account_id  BIGINT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
name        TEXT NOT NULL,
role        TEXT NOT NULL,
email       TEXT,
linkedin_url TEXT,
is_primary  BOOLEAN NOT NULL DEFAULT FALSE,
source      TEXT NOT NULL,
observed_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```
-- ——— Assets per account ———
```

```

CREATE TABLE account_assets (
id          BIGSERIAL PRIMARY KEY,
account_id  BIGINT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
asset_type  TEXT NOT NULL,    -- landing_page | video | email_template | embed
url         TEXT,
script_or_copy TEXT,
embed_code  TEXT,

```

```

created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
source        TEXT NOT NULL DEFAULT 'manual', -- manual | gan_ai_webhook
UNIQUE (account_id, asset_type)
);

```

```

-- — Signals (Phase 4+ ingestion) —

```

```

CREATE TABLE signals (
  id              BIGSERIAL PRIMARY KEY,
  account_id      BIGINT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  signal_type     TEXT NOT NULL,
  source          TEXT NOT NULL,
  title           TEXT NOT NULL,
  payload         JSONB NOT NULL,
  url             TEXT,
  observed_at     TIMESTAMPTZ NOT NULL,
  processed_at    TIMESTAMPTZ,
  fingerprint     TEXT NOT NULL,
  UNIQUE (account_id, fingerprint)
);

```

```

-- — Scoring runs —

```

```

CREATE TABLE scoring_runs (
  id              BIGSERIAL PRIMARY KEY,
  account_id      BIGINT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  total_score     NUMERIC(5,2) NOT NULL,
  max_score       INTEGER NOT NULL,
  tier             TEXT NOT NULL,
  raw_markdown    TEXT NOT NULL,
  structured_json JSONB NOT NULL,
  model           TEXT NOT NULL,
  cost_usd        NUMERIC(8,4),
  input_tokens    INTEGER,
  output_tokens   INTEGER,
  stop_reason     TEXT,
  parse_failed    BOOLEAN NOT NULL DEFAULT FALSE,
  triggered_by    TEXT NOT NULL, -- manual | signal | scheduled | bucket_change
  triggering_signal_id BIGINT REFERENCES signals(id),
  created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

-- — Engagement (the new heavy table) —

```

```

CREATE TABLE engagement_touches (
  id              BIGSERIAL PRIMARY KEY,
  account_id      BIGINT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  channel         TEXT NOT NULL, -- BOFU | TOFU | LinkedIn_Content | ... (matches weights table)

```

```

touch_count    INTEGER NOT NULL DEFAULT 1,
content_ref    TEXT,           -- "6 UM Trends 2026", "#1 Vanessa's email", etc.
source        TEXT NOT NULL,   -- linkedin_sales_nav | apollo | gan_ai | manual | ...
observed_at    TIMESTAMPTZ NOT NULL,
raw_payload    JSONB,
fingerprint    TEXT NOT NULL,
UNIQUE (account_id, fingerprint)
);

```

```

CREATE TABLE channel_weights (
  channel       TEXT PRIMARY KEY,
  weight        NUMERIC(5,2) NOT NULL,
  description    TEXT,
  updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_by    BIGINT REFERENCES users(id)
);
-- seed from xlsx legend; editable in UI by Galyna

```

```

CREATE TABLE intent_thresholds (
  id            BIGSERIAL PRIMARY KEY,
  min_score     NUMERIC(8,2) NOT NULL,
  max_score     NUMERIC(8,2),   -- NULL = open-ended (Hot)
  bucket        TEXT NOT NULL UNIQUE,
  action        TEXT NOT NULL,
  sort_order    INTEGER NOT NULL,
  updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_by    BIGINT REFERENCES users(id)
);

```

```

-- ——— Override + audit ———

```

```

CREATE TABLE tier_overrides (
  id            BIGSERIAL PRIMARY KEY,
  account_id    BIGINT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  scoring_run_id BIGINT NOT NULL REFERENCES scoring_runs(id),
  original_tier TEXT NOT NULL,
  override_tier TEXT NOT NULL,
  reason        TEXT NOT NULL,
  set_by_user_id BIGINT NOT NULL REFERENCES users(id),
  created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

CREATE TABLE account_status_changes (
  id            BIGSERIAL PRIMARY KEY,
  account_id    BIGINT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  from_status   TEXT,
  to_status     TEXT NOT NULL,
  changed_by_user_id BIGINT REFERENCES users(id),

```



```
    note          TEXT,
    created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

```
CREATE TABLE account_notes (
    id             BIGSERIAL PRIMARY KEY,
    account_id     BIGINT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
    user_id        BIGINT NOT NULL REFERENCES users(id),
    body           TEXT NOT NULL,
    created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

```
CREATE TABLE users (
    id             BIGSERIAL PRIMARY KEY,
    clerk_user_id  TEXT NOT NULL UNIQUE,
    email          TEXT NOT NULL UNIQUE,
    display_name   TEXT,
    role           TEXT NOT NULL DEFAULT 'analyst', -- analyst | owner | admin
    created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

```
CREATE TABLE audit_log (
    id             BIGSERIAL PRIMARY KEY,
    user_id        BIGINT REFERENCES users(id), -- NULL for system actions
    action         TEXT NOT NULL,              -- score | activate | override_tier | etc.
    entity_type    TEXT NOT NULL,
    entity_id      BIGINT NOT NULL,
    details        JSONB,
    created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

```
-- — Indexes —
```

```
CREATE INDEX idx_accounts_segment_tier      ON accounts(segment, current_tier);
CREATE INDEX idx_accounts_activated_owner  ON accounts(owner_id) WHERE is_activated;
CREATE INDEX idx_accounts_bucket           ON accounts(intent_bucket);
CREATE INDEX idx_accounts_status           ON accounts(current_status);
CREATE INDEX idx_signals_account_observed  ON signals(account_id, observed_at DESC);
CREATE INDEX idx_signals_pending           ON signals(processed_at) WHERE processed_at IS NULL;
CREATE INDEX idx_engagement_account_observed ON engagement_touches(account_id, observed_at DESC);
CREATE INDEX idx_engagement_channel        ON engagement_touches(channel);
CREATE INDEX idx_runs_account_created      ON scoring_runs(account_id, created_at DESC);
CREATE INDEX idx_status_changes_account    ON account_status_changes(account_id, created_at DESC);
CREATE INDEX idx_audit_user_action         ON audit_log(user_id, action, created_at DESC);
```

Tech stack — every box, opinionated (unchanged + refinements)

Layer	Tech	Why
Frontend	Next.js 15 (App Router) + TypeScript + Tailwind + shadcn/ui	Pairs with Claude Design output
Backend API	FastAPI + Pydantic v2	Async-native, OpenAPI-first
Database	Postgres 16 (Railway-managed)	Mid-scale fits; clean Snowflake path
Migrations	Alembic	Standard
Cache + Queue	Redis (Railway-managed)	Arq backing store
Background jobs	Arq	Async, light, Pydantic-friendly
Connector framework	Custom protocol + Arq jobs	One pattern for accounts + signals + engagement
Auth	Clerk (magic link + Google SSO)	Multi-user ready
LLM	Anthropic Claude Opus 4.7 + <code>web_search_20260209</code>	Validated
Hosting	All on Railway	One vendor
Observability	Sentry + structlog JSON to Railway log drain	Free at V1 scale
Python tooling	uv + ruff + mypy + pytest	Astral stack
JS tooling	pnpm + vitest + playwright + biome	Rust toolchain
CI	GitHub Actions	Matrix lint/type/test/build
PDF generation	weasyprint (server-side, no Chrome)	Removes Chrome dep from production

Why weasyprint over headless Chrome: at production scale, spawning Chrome per PDF is fragile (memory, race conditions). Weasyprint is a Python lib, runs in the Arq worker, deterministic output. We give up some Chrome-only CSS but the report templates don't need it.

Connector framework (the heart of multi-input)

All external inputs — account sources, signal sources, engagement sources — implement one protocol:

```
class Connector(Protocol):
    name: str
    kind: Literal["account", "signal", "engagement"]
    schedule: str | None # cron-ish, e.g. "* /15 * * * *" or None for webhook-only
    def poll(since: datetime) -> Iterable[Record]: ...
    def webhook_handler(payload: dict) -> Iterable[Record]: ... # optional
```

```

class Record(BaseModel):
    account_match_hint: str | dict # name/domain for account-resolver
    payload: BaseModel            # connector-specific Pydantic model
    fingerprint: str              # dedupe key
    observed_at: datetime

```

One connector module per source. Adding a new source = drop a new module, register, run migration to add weights/configs. No core code touched.

Connector failure isolation: if Apollo connector breaks, Apify connector keeps running. Each connector has its own Arq queue with retries + dead-letter.

V1 ship order: Definitive (Stage 1 account) first; engagement connectors stack on as Sunny gets API access.

Production-grade requirements (refined)

Category	Practice	Concrete implementation
Repo	Monorepo (<code>apps/api</code> + <code>apps/web</code> + <code>packages/shared-types</code>)	pnpm workspaces + uv
Type safety	Pydantic v2 at every API boundary; OpenAPI → TS client	openapi-typescript on prebuild
Testing	Unit (pytest + vitest), integration (httpx + testcontainers Postgres), smoke E2E (playwright on triage + score + activate flows)	Coverage ≥ 70% on core domain
Lint + format	Ruff (Python), Biome (JS/TS)	Pre-commit + CI
Type checking	mypy strict; tsc strict	CI gate
CI/CD	GitHub Actions: lint → type → test → build on every PR; auto-deploy main → Railway	Branch protection on main
DB migrations	Alembic with autogenerate + manual review; one migration per PR	Required PR check
Secrets	Railway env vars; rotation runbook in docs/RUNBOOK.md	Annual rotation reminder
Auth	Clerk middleware on every API route except <code>/health</code> , <code>/ready</code>	Audit log on state changes
Error tracking	Sentry for unhandled exceptions on api + web + worker	Release tagging on deploy
Structured logs	JSON with trace IDs (W3C traceparent)	structlog + pino
Rate limiting	Per-endpoint (<code>fastapi-limiter</code>); per-LLM-call MTD cost guard	Hard-stop at \$500/mo MTD
API versioning	<code>/api/v1/...</code> from day 1	Routes namespaced

Category	Practice	Concrete implementation
Background jobs	Arq with retries (3, exponential backoff), dead-letter queue, deterministic job IDs for idempotency	Per-connector queue
Idempotency	All mutating endpoints accept <code>Idempotency-Key</code> header; engagement touches deduped by fingerprint	Middleware
Health + readiness	<code>/health</code> (cheap), <code>/ready</code> (DB + Redis + Anthropic check)	Railway healthcheck
Feature flags	Env-var-driven flag map (<code>FLAG_*</code>); revisit LaunchDarkly only if >5 flags	Custom
DB backups	Railway managed daily snapshots, 7-day retention	Documented in RUNBOOK
Cost guardrails	Per-run cost logged; MTD alert at \$100 (Sentry), hard cap at \$500 (config)	Env-configurable
Docs	OpenAPI auto-generated; README + RUNBOOK + DECISIONS (ADRs)	One ADR per non-obvious choice
Repo hygiene	Conventional commits; squash-merge; release-please for CHANGELOG	Automated

Integration map

Source / sink	Stage it serves	Mechanism	Phase
Anthropic API	2 (done)	REST + streaming	1
Definitive Health	1	REST API	2
Becker's IT lists	1	CSV upload (UI)	2
PRI Congress / APTA lists	1	CSV upload (UI)	2
Apollo Sequences	5 (Email engagement)	REST API + webhook	4
LinkedIn Sales Navigator / Company API	5 (LinkedIn Content + Connect)	REST API or Apify scraping fallback	5
gan.ai	5 (LP Video View)	REST API + webhook	5
LP hosting (TBD which platform)	5 (LP Visit)	Analytics API	5
Buzzsprout / podcast platform	5 (Podcast Guest)	API or manual flag	6
Event platform	5 (Event Attend)	API or manual flag	6

Source / sink	Stage it serves	Mechanism	Phase
Salesforce	7 (Funnel sync)	REST API	7 (V2 candidate)
Slack	4 / 6 (notifications)	Webhook	4
Snowflake	(V2 BI)	dbt + Fivetran	V2
Omni	(V2 BI)	Connect to Snowflake	V2

Phase plan (concrete, post-Vision)

Each phase ships independently. CLI keeps working at every step. No big-bang rewrite.

#	Phase	Outcome	Effort
1	Foundation refactor	Modular <code>apps/api</code> package, Pydantic models for all 10 tables, Postgres + Alembic, Arq worker, structured logging, scoring engine lifted from CLI	2 sessions
2	Web app skeleton	Next.js + Clerk auth + FastAPI scaffold + monorepo CI + Sentry + Railway deploy	2 sessions
3	Universe + Scoring + Score-New UI	Definitive API connector, CSV upload for Becker's/PRI/APTA, Triage view (basic), Score-New form with live streaming, Account Detail page	3 sessions
4	Activation + Owner routing + Slack	Activation flow, owner assignment, status transitions, Slack DM on activation + bucket changes	1 session
5	Engagement: Apollo (email)	First engagement connector — Apollo API, touch ingestion, weighted compute, intent bucketing	2 sessions
6	Engagement: LinkedIn + gan.ai + LP analytics	Three connectors in parallel (or sequence as APIs allow)	3 sessions
7	Funnel + Reporting (P0)	Status transitions, owner pipeline view, attribution dashboard, value-based outcomes	2 sessions

#	Phase	Outcome	Effort
8	Polish + ops	Cost dashboard, backup verification, RUNBOOK, monitoring alerts, V2 handoff	1 session

Total: ~16 sessions to ship V1-complete. Each phase has a demo-able outcome.

Critical path bottleneck: Stage 5/6 engagement connectors all depend on API access Sunny is sourcing. Without API access, those phases stall — engagement stays empty, but the rest of the system works (manual touches via admin UI as escape hatch).

Open dependencies (blockers to unblock)

Item	Blocks phase	Owner	Status
Definitive Health API key + docs	3	Sunny / Galyna	Sunny chasing
Anthropic API key under Magical org	Operational	Galyna / IT	Pending
Clerk free-tier signup + Google Workspace SSO config	2	Sunny	Quick
Railway project setup (Postgres + Redis)	2	Sunny	Quick
Sentry free account	2	Sunny	Quick
API access list for engagement connectors (Apollo / LinkedIn / gan.ai / LP analytics / podcast / event platform)	5–6	Sunny	Sunny sending
Domain (<code>abm.magical.com</code> ?)	2	Galyna	Optional V1
One-time xlsx → DB migration script (seed accounts + assets)	3	Sunny	Will write during Phase 3
Salesforce read access (for <code>is_customer</code> flag)	1 (nice-to-have)	Galyna	Defer to V2 if delayed

Out of scope (V1)

Explicit list of what V1 does NOT do, so feedback lands somewhere instead of bloating scope:

- ✗ LP / video generation (URLs stored, generation manual)
- ✗ Auto-routing of Account Owners (Galyna picks; suggestion engine is V2)
- ✗ Salesforce write-back (status changes don't sync to SFDC)
- ✗ Outbound email / LinkedIn sequence automation (V1 reads engagement, V2 writes outbound)

- ❌ Notion auto-push (app is source of truth)
 - ❌ Snowflake / Omni dashboards (V2 BI layer)
 - ❌ Multi-tenant (single Magical instance)
 - ❌ Mobile UI
 - ❌ Multi-segment accounts
 - ❌ Public API
 - ❌ Direct mail tracking automation (manual flag entry; no API exists)
 - ❌ Weekly digest emails (V2)
 - ❌ Audio podcast tracking automation (manual flag for V1)
-

QA / validation (designed open-ended, per earlier decision)

The schema supports these audits without code changes — built into reports as they become needed:

- **Score drift** — `scoring_runs` history per account; alert if Tier 1 → <Tier 1 in successive runs
- **Override frequency** — `tier_overrides` aggregated per segment exposes prompt misalignment
- **Parse failures** — `scoring_runs.parse_failed=TRUE` percentage trended weekly
- **Connector health** — last successful poll time per connector; alert if no touches in 24h on a previously-active connector
- **Cost anomaly** — per-account cost outliers (model regression vs. baseline)

Phase 9 (post-V1) builds dashboards on these. Phase 1 ensures the data exists.

Glossary

- **Account** — a company we're targeting. One segment. Persistent.
 - **Contact** — a person at an account.
 - **Asset** — landing page / video / email template tied to an account (V1: URL only; V2: generated).
 - **Signal** — observed external event (leadership change, funding round, EHR migration).
 - **Engagement Touch** — observed first-party event (LinkedIn engagement, email open, LP visit).
 - **Scoring Run** — one Claude execution. Versioned, audited.
 - **Tier** — Claude's fit verdict (High Fit / Medium / Low / Tier 1–4 for HS).
 - **Intent Bucket** — engagement-driven bucket (Lower / Some / Warm / Hot).
 - **Action** — the suggested next step per intent bucket.
 - **Activation** — Galyna's decision to commit resources; assigns Owner; transitions account from COLD to ACTIVATED.
 - **Owner** — Account Owner (SDR/AE) responsible for the activated account.
 - **Funnel** — the status chain from ACTIVATED → WON/LOST/PARKED.
 - **Triage** — Galyna's daily/weekly review surface.
 - **Connector** — pluggable input module (account/signal/engagement source).
-

What to do next

1. **Galyna review** of this doc — confirm priorities, segment ordering, engagement weights, intent thresholds
 2. **Sunny sends API access** — Apollo, LinkedIn, gan.ai, LP analytics. Unblocks Phase 5/6.
 3. **Claude Design briefing** — paste the *Stage 4–8 UI specs* into Claude Design for screen mocks
 4. **Start Phase 1** — foundation refactor (see Phase Plan table above)
 5. **Open issues:** convert "Open dependencies" into tracked items (GitHub issues or Notion)
-

ADRs to write (one per non-obvious decision, ~150 words each)

1. Postgres over SQLite (concurrent writes + Snowflake portability)
2. Arq over Celery (async-native, lighter)
3. Clerk over self-rolled auth (multi-user upgrade path)
4. Monorepo over polyrepo (shared types, single deploy)
5. uv + ruff + biome (Astral stack)
6. One segment per account (vs many-to-many)
7. PDFs regenerated on demand (not stored)
8. Tier override appended, never overwritten (audit integrity)
9. Asset generation OUT of V1 (URLs stored only; manual pipeline continues)
10. Engagement = API-only, no manual UI form (no spreadsheet behavior in V1)
11. Channel weights + intent thresholds = config tables, not constants (Galyna-tunable)
12. Connector framework = one protocol for all input sources (extensibility from day 1)
13. WeasyPrint over headless Chrome (production reliability)
14. Reports rendered server-side from Postgres views (no BI layer in V1)